

We start at 1.05

Material for this workshop

Workshop materials

- Slides
- Hands-on data and scripts

Please drop your questions in the chat (to everyone) or unmute yourself

GLADSTONE
INSTITUTES

Machine Learning using R

Michela Traglia
Ayushi Agrawal

September 24th, 2026

Introductions

Michela Traglia

Associate Core Director

Ayushi Agrawal

Senior Bioinformatician

Poll1 . Have you ever used any of these methods? (Please select all that applies)

1. Clustering
2. Principal Component Analysis
3. UMAP/tSNE
4. Linear or logistic regression
5. Random Forest
6. None of the above

What to expect from this workshop

Our aim is to give a basic overview of the logic behind machine learning methods and how they work.

We will go over some questions that you may have:

- What are the differences between AI, ML, DL?
- Which are the types of machine learning?
- Which are the steps to build a prediction model?
- What is overfitting in ML?
- How can you evaluate the prediction model? Which metrics should I use?

In our demo session you will practice how to work with a real dataset and two simple ML algorithms in R

How Machine Learning can inform biomedical research

[nature](#) > [nature methods](#) > [articles](#) > [article](#)

Article | Published: 12 October 2020

Predicting 3D genome folding from DNA sequence with Akita

[Geoff Fudenberg](#) , [David R. Kelley](#)  & [Katherine S. Pollard](#) 

[Nature Methods](#) **17**, 1111–1117 (2020) | [Cite this article](#)

26k Accesses | **295** Citations | **462** Altmetric | [Metrics](#)

<https://www.nature.com/articles/s41592-020-0958-x>

Input: ~1 million base pairs (1 Mb) of **DNA sequence**

Training based on experimental chromatin conformation capture data — Hi-C and Micro-C contact maps — from human cell lines - contact frequency between bp

Method: Convolutional neural network

Output: a **3D contact map** - a matrix showing **predicted** contact frequencies between all pairs of 2,048-bp regions within that window.

How Machine Learning can inform biomedical research

Deep learning systems developed by **DeepMind**

Predict 3D protein structures from amino acid sequences.

Trained on ~200,000 experimentally solved protein structures

Most recent database release contains over 200 million entries

The image shows the AlphaFold Protein Structure Database website. It has a blue background with a faint protein structure. The title 'AlphaFold Protein Structure Database' is in large white text. Below it, it says 'Developed by Google DeepMind and EMBL-EBI'. There is a search bar with the placeholder text 'Search for protein, gene, UniProt accession or organism or sequence search' and a 'Search' button. Below the search bar, there are examples: 'MENFQKVEKIGETYG...', 'Free fatty acid receptor 2', 'At1g58602', 'Q5VSL9', and 'E. coli'. At the bottom, there are links for 'See search help', 'Go to online course', and 'See our updates - November 2025'.

Developed by Google DeepMind and EMBL-EBI

Search for protein, gene, UniProt accession or organism or sequence search

Search

Examples: MENFQKVEKIGETYG... Free fatty acid receptor 2 At1g58602 Q5VSL9 E. coli

See search help

Go to online course

See our updates - November 2025

AlphaFold DB provides open access to over 200 million protein structure predictions to accelerate scientific research.

<https://www.nature.com/articles/s41586-021-03819-2>

September 8, 2026 Science

AlphaGenome Atlas: A predictive map of every possible DNA letter change in the human genome

AlphaGenome Atlas team

Predict genomic function from sequence

Outline

- Basic concepts of AI, ML, DL
- Unsupervised and Supervised Machine learning definition
- Supervised machine learning workflow
 - a. Features selection
 - b. Concepts of bias/variance, and over/underfitting
 - c. Hyperparameters
 - d. Model evaluation
- Algorithms overview

Hands-on on supervised and unsupervised ML methods

AI universe - basics

Artificial Intelligence (AI)

Systems that can simulate human intelligence

Machine Learning (ML)

Systems that learn from data

Deep Learning (DL)

Neural networks with multiple layers

NLP

Processing human language

Generative AI

Creating new content

AI definition and examples

- Task-specific algorithms
- ***Rules-driven (IF-THEN)***
- Perform human preset tasks
- Needs clear logic - limited flexibility

Applications:

- Early chess game
- Speed limit enforcement systems

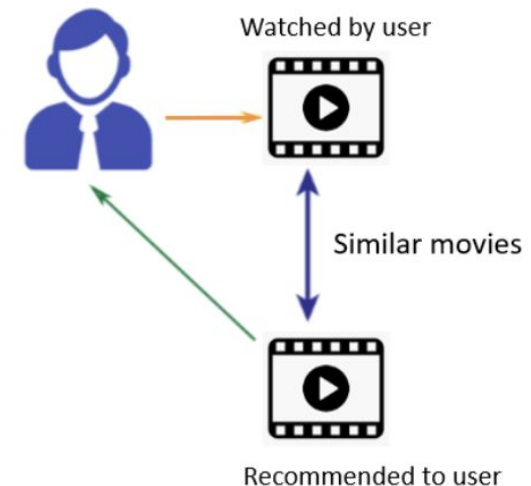


ML definition and examples

- Uses algorithms trained on data sets to learn patterns -> create models capable of predicting outcome
- ***Data-driven***
- Improve over time without being programmed
- Easy to interpret

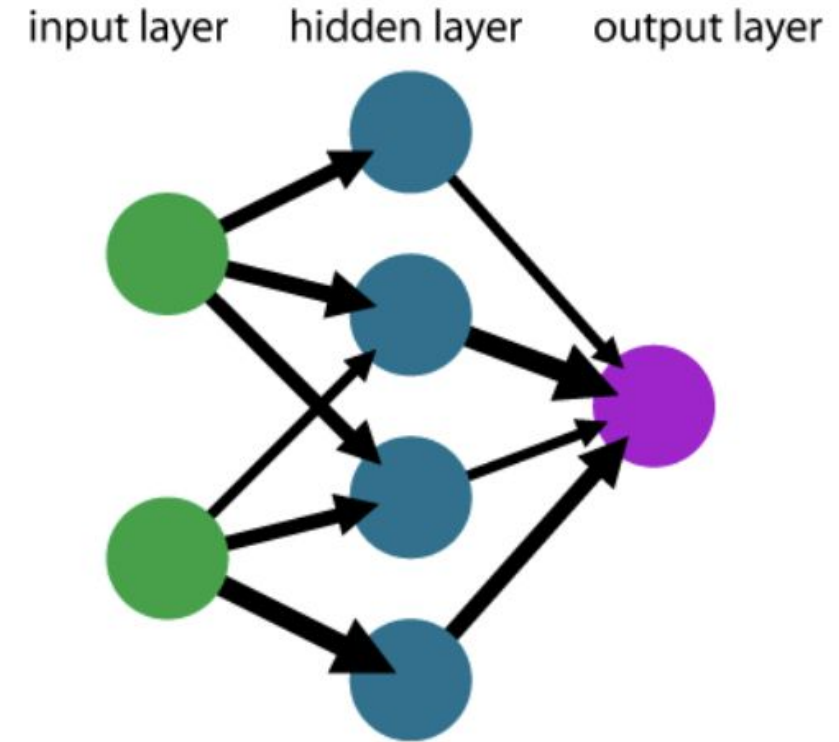
Applications:

- clinical prediction
- virtual assistant
- movie recommendation



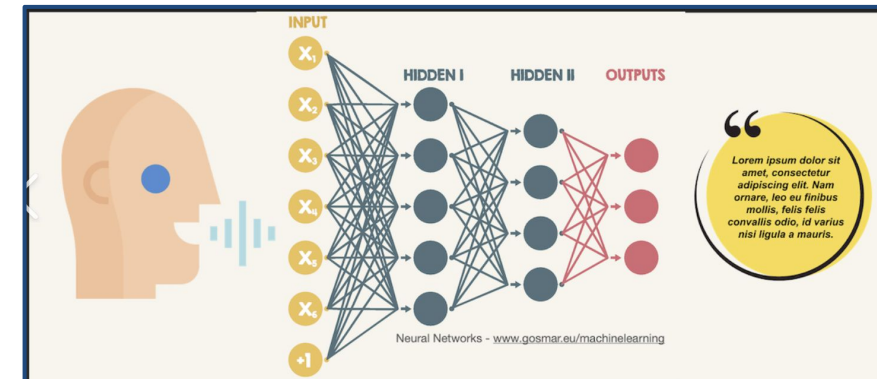
Deep learning definition and example

- Mimic the human brain
- More complex problems - more data needed
- Select features and recognize patterns
- ***Relationship-driven***
- Adapt the strength of the connections based on the learning of the information



Applications:

- Image recognition
- Natural language processing (NLP) to understand language



Generative AI and Large Language Model (LLM) - examples

- Patterns -> generate new content (image, text, code)
- LLM is NLP+GenAI
- Use the neural network trained on massive amounts of text
- Understand a sentence and predict next word (human-like language)
- Based on *attention* - importance of every word in relation with all the other words to build a meaningful sentence

Applications:

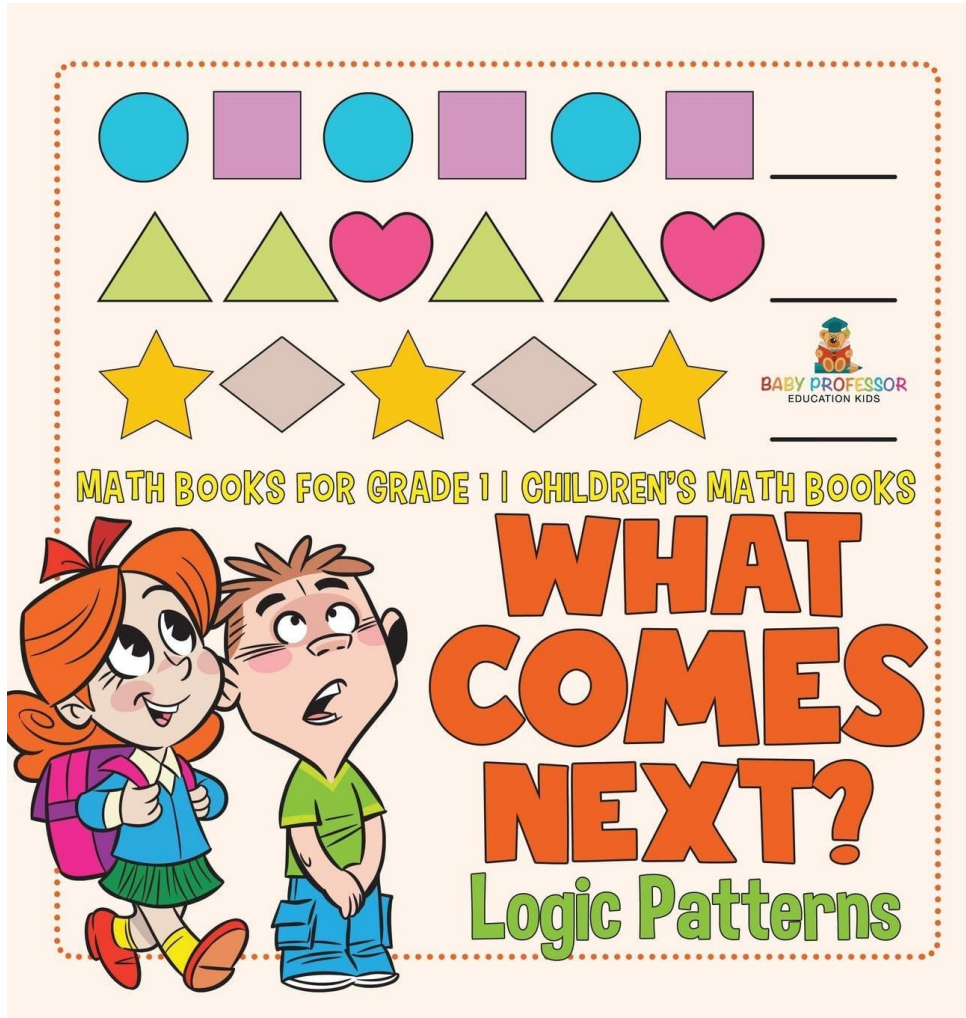
- GPT, Claude, Google gemini

AI - Core position paper



Machine Learning

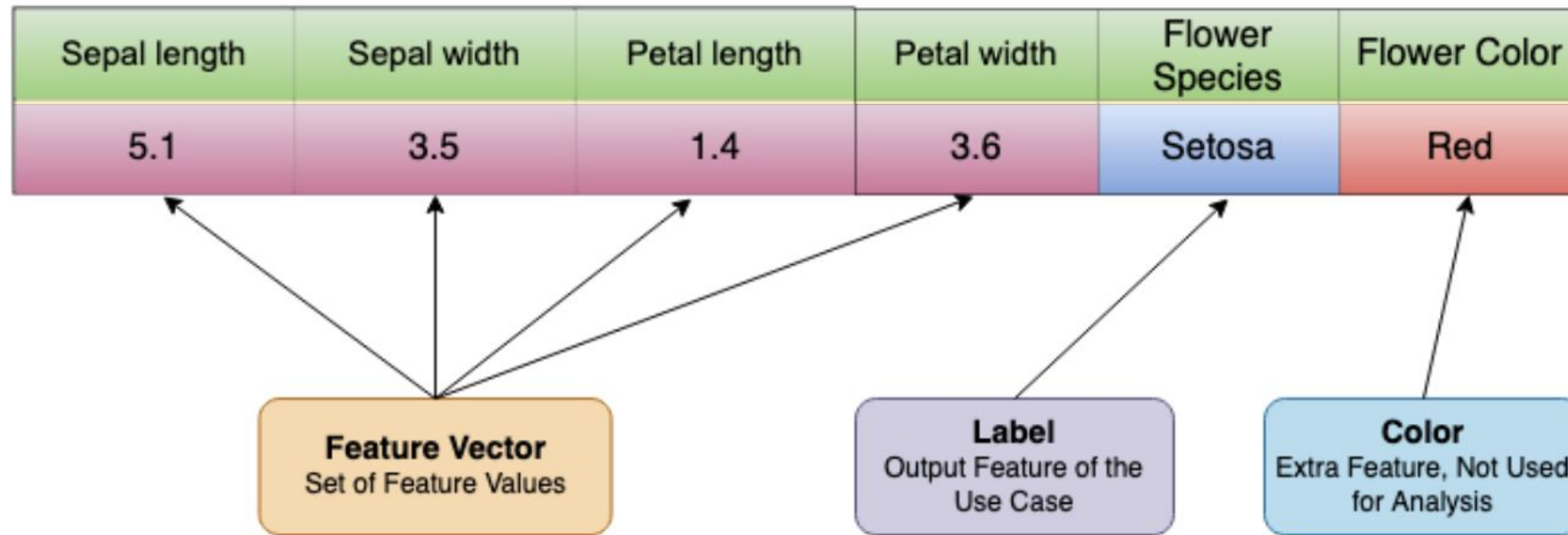
Aim of ML



Find patterns across a set of variables or **features** with the aim to **define groups** or **predict a label**

Input space

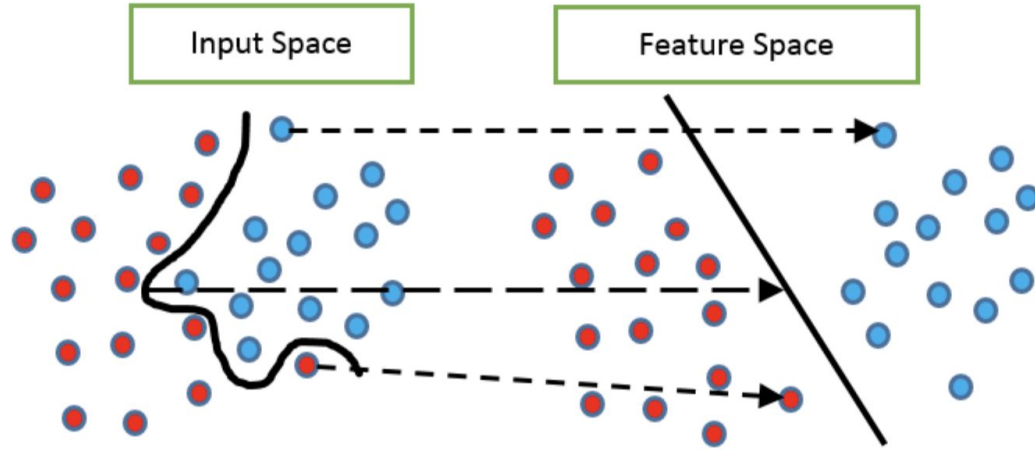
- Data points that we need to group or classify
- Collected features, characteristics, biological measures, variables for each data point
- Feature vector is an **n-dimensional vector of values of features** that represent the data points



- Input space -> transformed space where each **dimension corresponds to a feature** useful for learning (scaling the data, ...)
- Easier to capture the underlying patterns in the data

Feature space

- The multidimensional space formed by **all input features** (variables) of a dataset
- Each data point is a vector in this space



More features -> need more data to reliably learn patterns

Rule of thumb: **Sample size should grow exponentially with the number of features**
Sample size (10-30 times) >> number of features
More complex model -> more samples

Small datasets in high-dimensional spaces -> overfitting risk.

In biomedical research we aim to **define groups** of patients, mice, etc...given some biological characteristics

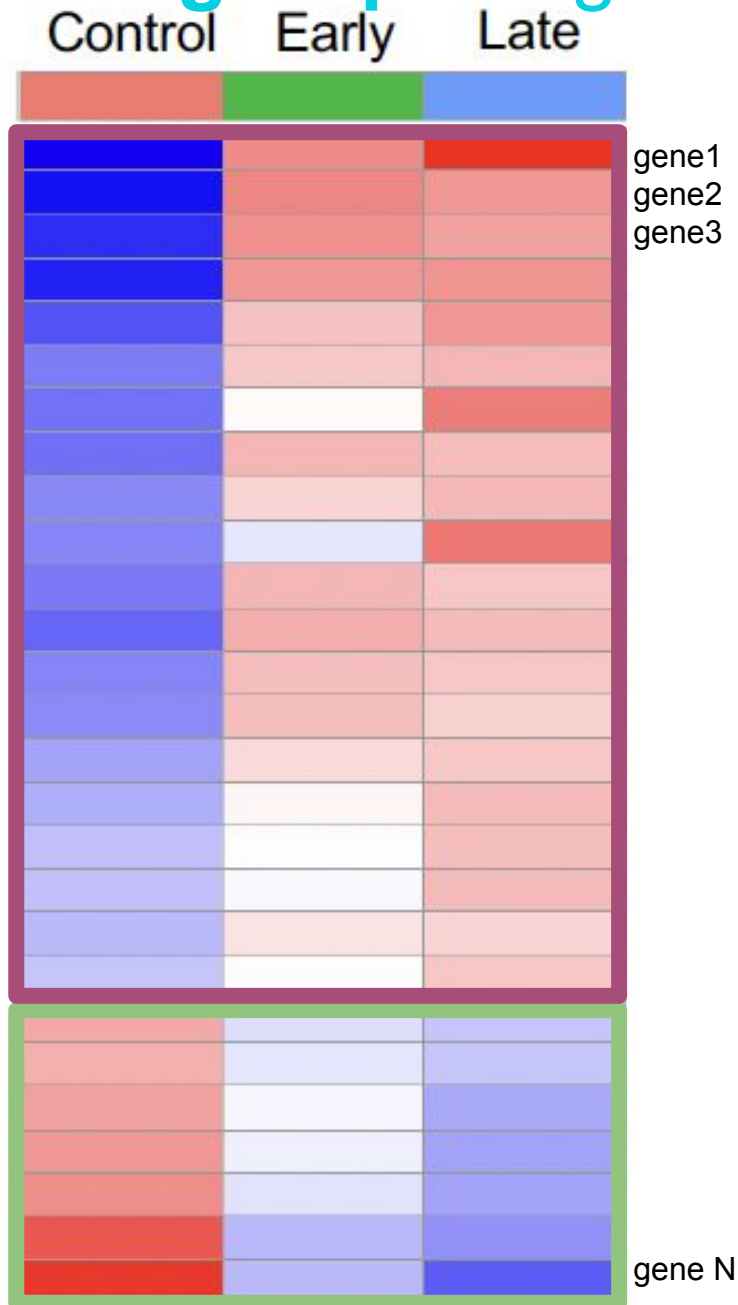
features	c1	c2	c3	Groups
Patient 1				1
Patient 2				2
Patient 3				3
Patient 4				2
Patient 5				1

c1, c2, c3 are the **features**, (ex. gene expression of 3 genes)

c1 = high gene1 expression 
low gene1 expression 

Find patterns across characteristics (features) -> subgroups of patients

Find groups of genes with similar gene expression by condition



- Analyse gene expression in control, early disease and late disease patients
- Find patterns within each group
- Find patterns between groups
- Heatmap comparing gene expression changes in control, early disease and late disease groups

We identified two main clusters of genes with opposite gene expression trends across the three groups of interest

We want to **predict an outcome** based on a set of **characteristics**

features	c1 c2 c3	Disease	labels
Patient 1	  	yes	
Patient 2	  	no	
Patient 3	  	no	
Patient 4	  	no	
Patient 5	  	yes	

The features c1, c2, and c3 can be categorical or continuous variables

c1 = smoking Y  smoking N 

- **Find** patterns across characteristics (features)
- **Learn** the relationship with the label
- **Predict** the outcome on a new dataset

Features = Blood biomarkers levels -> outcome = disease/cancer

Machine Learning categories

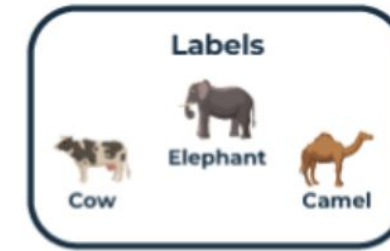


- **Unsupervised:**

Find patterns of **unlabeled data** (unknown categories/outcome) and subgroup

- **Supervised:**

Find a model that maps input data to **labeled data** (known categories) and predict outcomes in new dataset



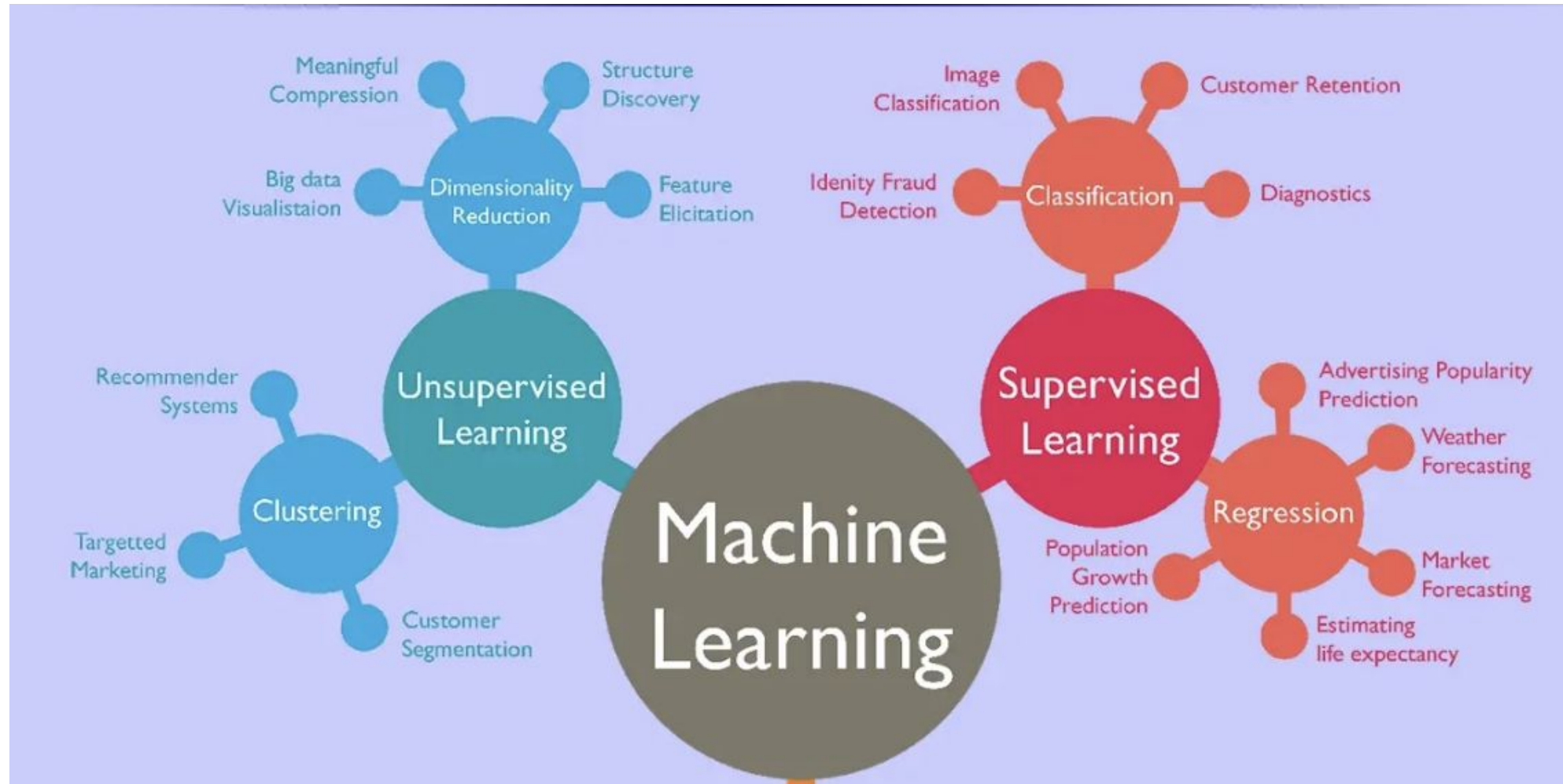
- **Semi-supervised:**

Use a small group of labeled data (some known outcomes) to learn and to predict a large group of unlabeled data

- **Reinforcement Learning (RL):**

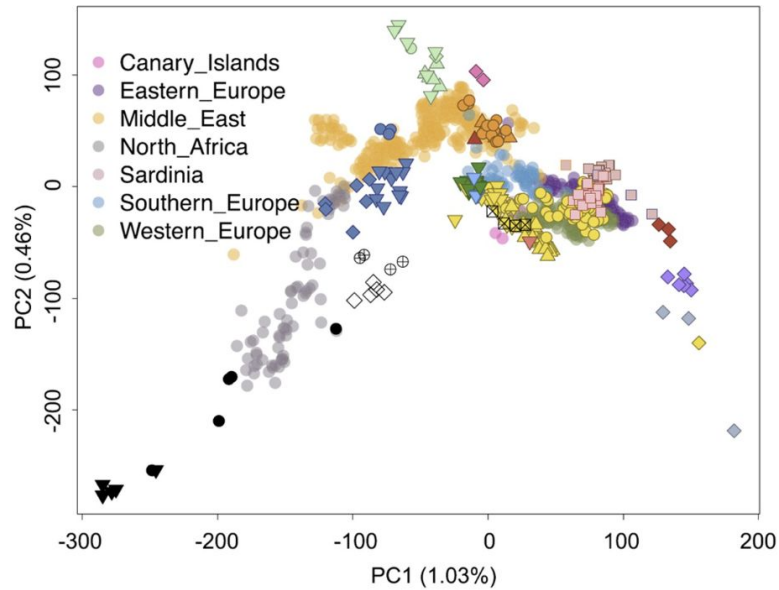
A system learn how to interact with the environment - observe, action, award and penalties

We focus on two main Machine Learning approaches

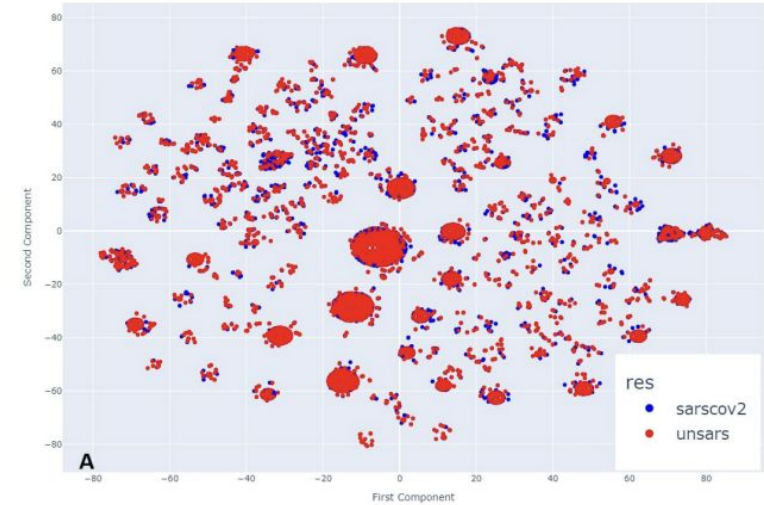


Unsupervised methods: the machine finds the hidden structure of the data

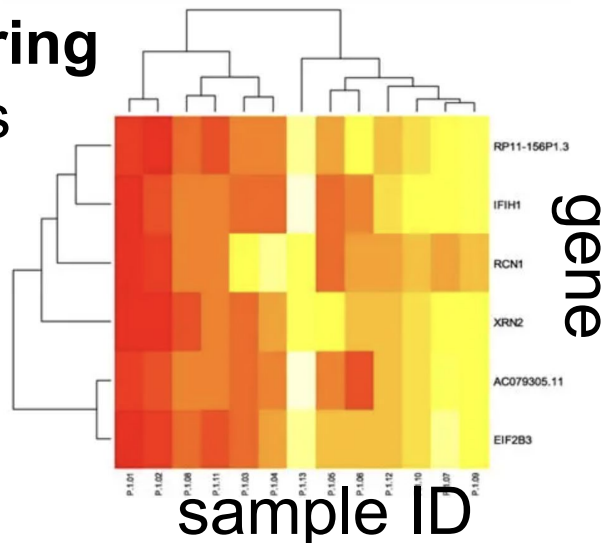
PCA - genetic ancestry



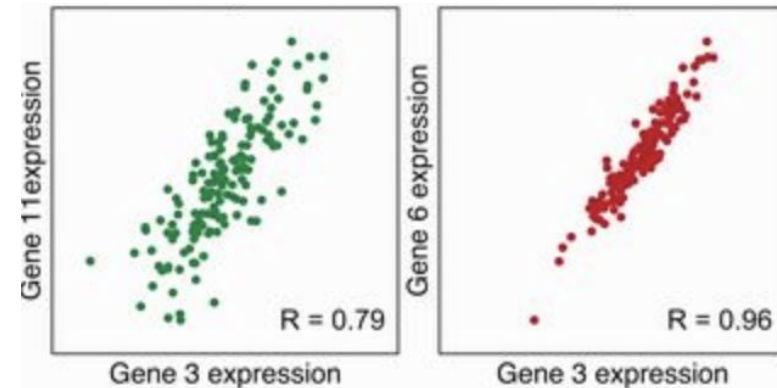
K-means clustering of patients showing similar symptoms of SARS-COVID19



Hierarchical clustering of heatmap of DEGs



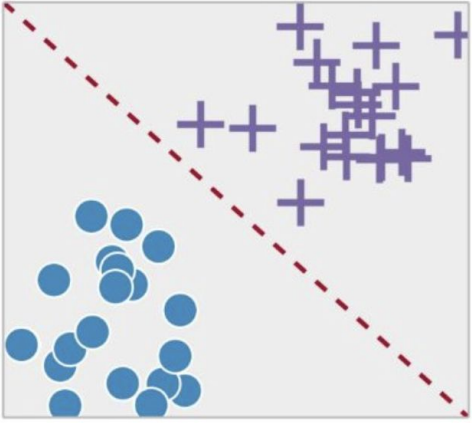
Association - find co-occurrence of gene expression and define groups



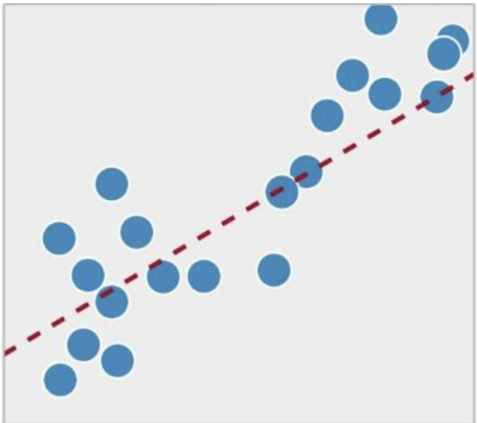
gene 3+6= D1
gene 3+11=D2

Supervised learning approaches

Classification - Model features to best predict **categorical outcomes** (disease affected vs unaffected or disease subtypes)



Regression - Model features to best predict **continuous traits** (human biomarker levels, mice histological measures)



Regression models for association analysis vs ML

Association analysis:

*Is this **SNP** significantly associated with **levels of cholesterol**?*

*Is **age** and **BMI** associated with **thyroid dysfunction**?*

- **Response variable** (outcome/trait) and the **predictor variable/s**
- Find the **association** between these two variables
- Find the effect of the **predictor** on a the **response** variable
- Estimate the statistical significance

ML analysis:

How well a set of features (age, BMI) can predict the thyroid dysfunction in a independent set of patients?

- Find the best **prediction**

Poll 2. I performed a knockout experiment. I want to cluster genes based on gene expression similarity.

What approach will you apply?

1. Supervised
2. Unsupervised

Poll 3. We have measured the *brain volumes* and *brain gene expression* in a group of mice.

We want to predict the ***brain volume measures*** using mice *gene expression* levels.

Which model will you apply?

1. Supervised regression model
2. Supervised classification model

Supervised machine learning analysis workflow

1. Define your biological question and what you want to predict

Classify individuals with a disease

Classify patients with a disease subtype

...

- **What is important? What would you like to minimize in your prediction?**
 - individuals wrongly classified as affected / wrongly classified as non affected?
 - patients wrongly assigned to one disease subtype?

Costs, invasive diagnosis, therapies...

2. Which algorithm works best for my question and data type?

- Each algorithm fits the data in a different way
- Each model may work best depending on your questions
- Consider apply more than one method and compare (e.g. *SuperLearner* function in R applies multiple approaches)

3. Training step. How much of the data to train the model?

features

c1	c2	c3	D
----	----	----	---

labels

Patient 1				yes
Patient 2				no
Patient 3				no
Patient 4				no
Patient 5				yes

	c1	c2	c3	D
Patient 1				yes
Patient 2				no
Patient 3				no

Labeled Training set



	c1	c2	c3	D
Patient 4				no
Patient 5				yes

Labeled Test set



yes

All dataset? -> how to evaluate the prediction on new data?

The dataset will be **split** in two sets - training (majority of the samples) and test dataset

Important! No same subjects in the training and test set or model overestimation

Ex. Training 70% - Test 30% or Training 80% - Test 20%

Be careful when splitting **imbalanced** case/controls data sets

Be careful with missing data - consider imputation

The model learn on the training set -> save the model -> predict the outcome of the labeled test set

How can we generate a prediction model that is 'good enough'? How can we evaluate 'good enough'?

- **Selecting** the right set of features
- **Fitting** the right model on the training
- **Tuning** the algorithm parameters
- **Evaluating** the model performance based on your expectation -> change the model accordingly

4. Feature selection

Biological question: *I want to predict IVF success rate (label)*

Which features may be the most useful?

mother age

number of baseline follicles

sperm motility

mother BMI

donor/partner age

mother height

maternal ancestry

alcohol consumption

Data leakage

Biological question: I want to predict IVF success rate (label)

In the predictive model, can I use the feature **‘child weight’**?

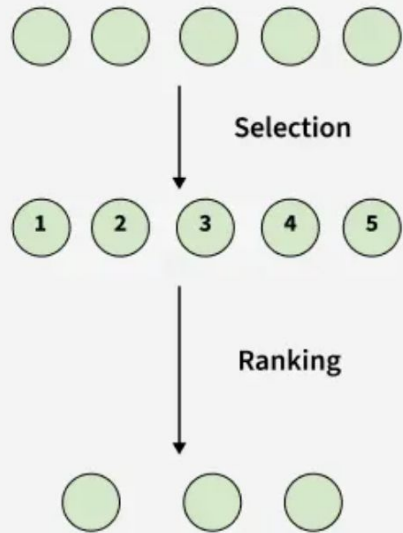
Information from outside the training data accidentally influences the model during training

Incorrect use data that are related to the outcome to predict

Predict if a student pass the exam

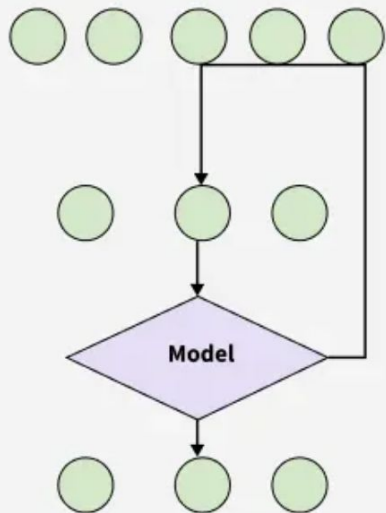
Include **‘hours of study per day’** not the **‘grade’**

Feature selection methods and advantages



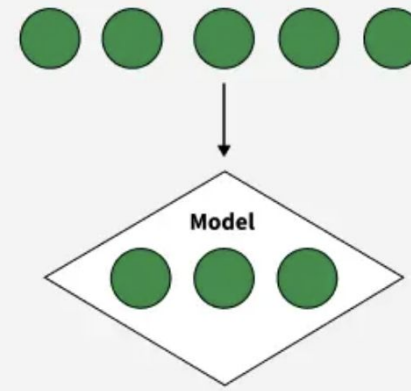
Filter Method

- 1 Rank features by statistical score
- 2 Select top-ranked features
- 3 Feed into model



Wrapper Method

- 1 Select a subset of features
- 2 Train model and evaluate
- 3 Repeat with different subsets



Embedded Method

- 1 Train model with all features
- 2 Model selects important features
- 3 Use selected features

- Learn on only important features
- Reduce computation time
- Easier understanding of the model behavior
- Reduces complexity with high-dimensional data

Break

See you in 5 minutes

5. Fitting the Training model

Good prediction model = good fitting of your data

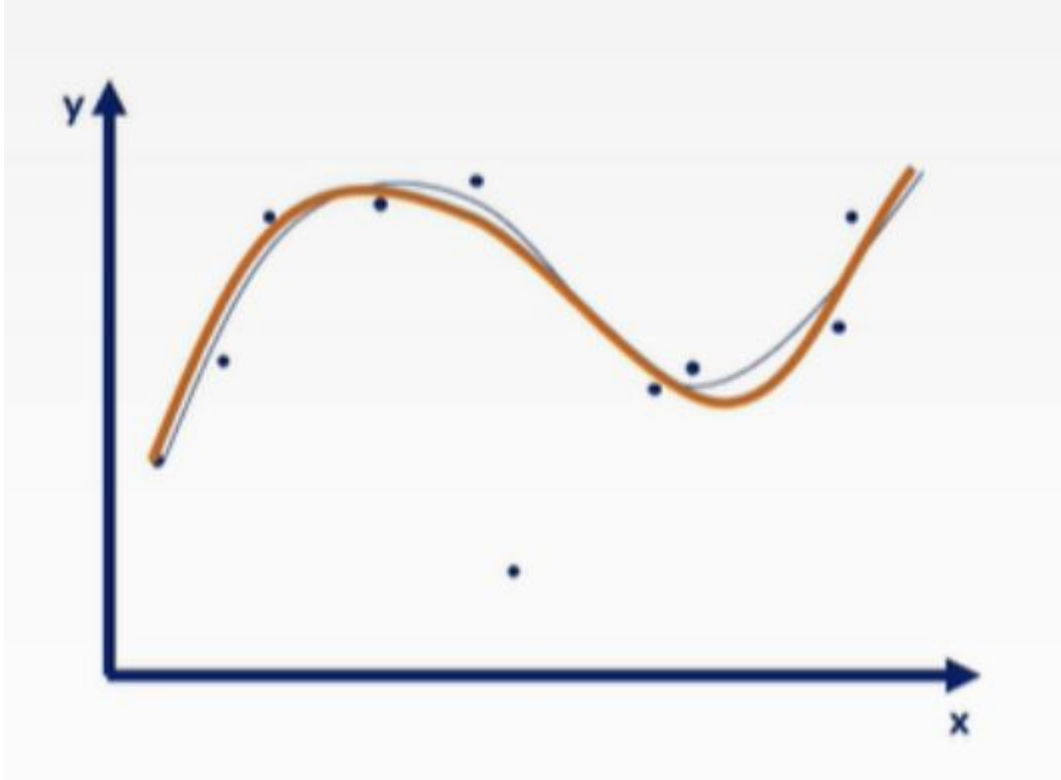
Balance of Bias and Variance



Bias: model fails to capture the true relationship between input and output

Variance: model learns noise from the data and fails to generalize

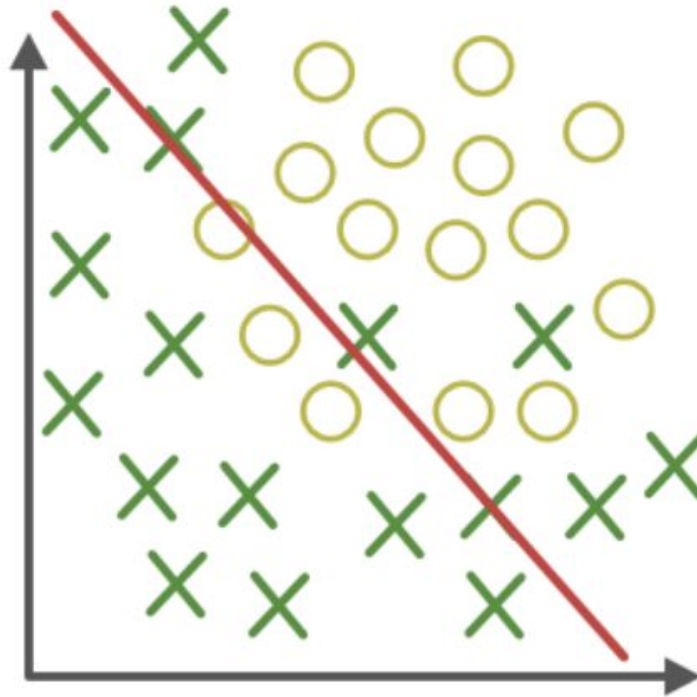
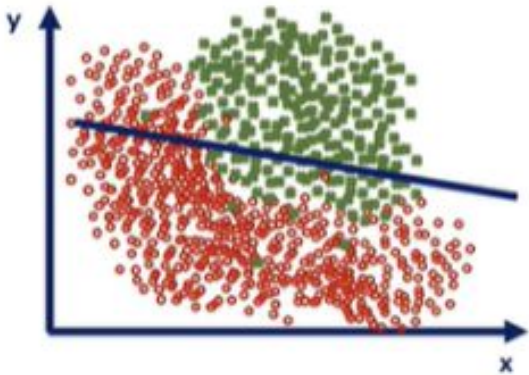
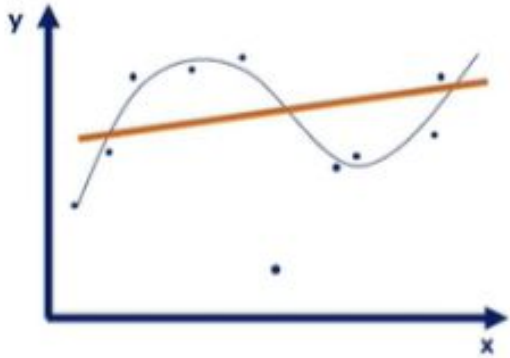
Good fitting



 fitting

High bias (low variance)

Model too simple, **underfitting** ->
poor performance on the training and on the test



Tips:

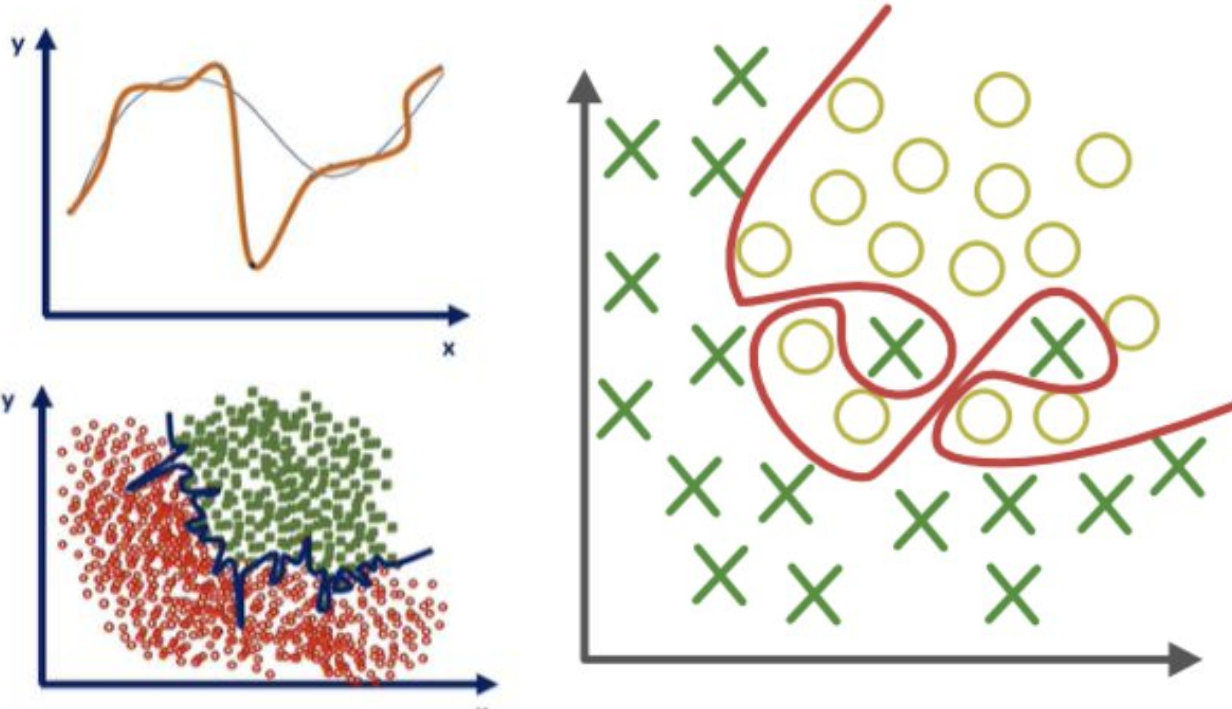
- Add more informative features / interaction terms
- More complex model

High variance (low bias)

Model fit too close the data capturing noise, **overfitting**

Great performance on the training set

Poor performance on the test/not generalizable



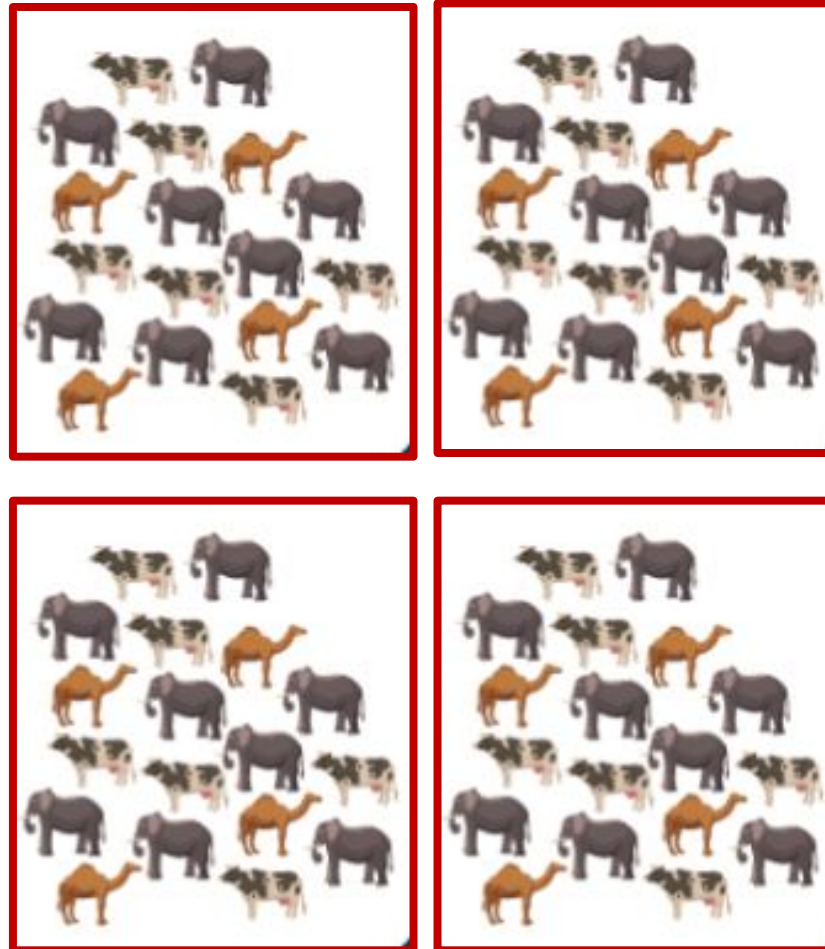
Tips:

- Remove features from the model
- More data

How can we select a model that balances variance & bias?

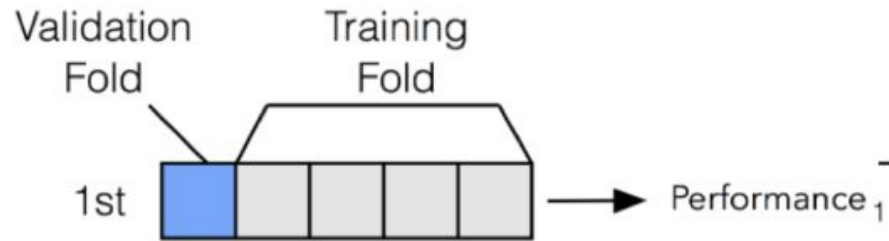
Cross Validation

- k-Fold CV



TRAINING SET split
in K folds

Cross validation is an iterative process

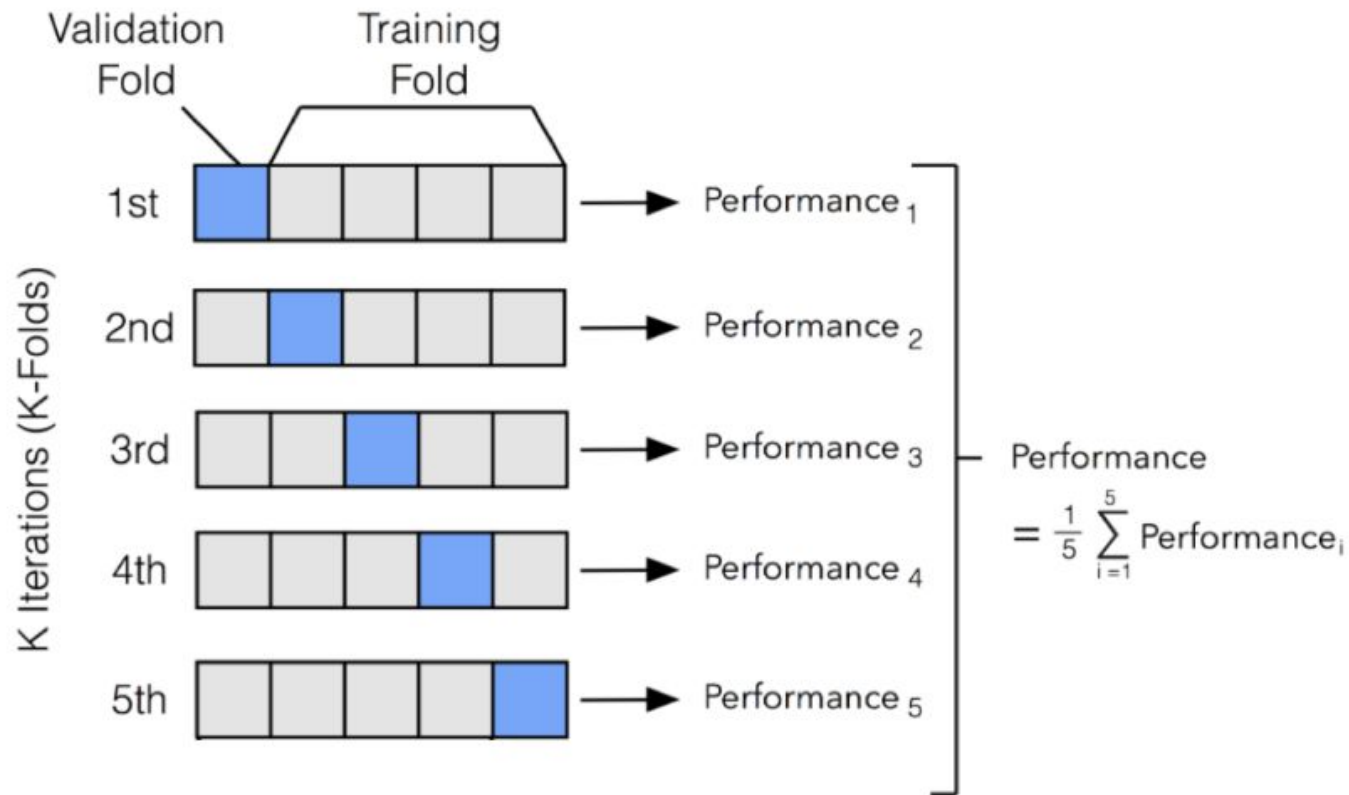


Iteration 1



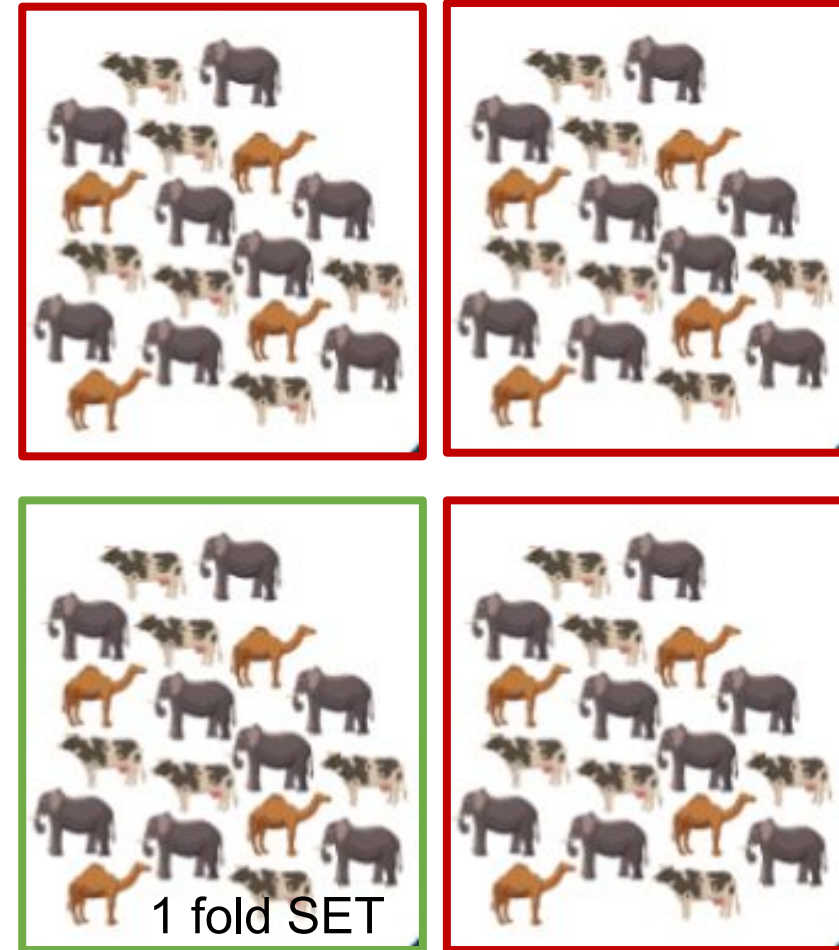
K-1 folds **TRAINING SET**

Cross validation is an iterative process



- The model is trained and tested on **multiple data splits**
- Bias and variance can be balanced

Iteration 2



K-1 folds **TRAINING SET**

Cross validation approaches and R packages

- k-Fold CV
- Leave-One-Out CV - maximizing the data training
- Repeated k-Fold CV - repeat n times the k folds random selection
- Stratified k-Fold - keep proportion of each class

Several R packages:

[caret](#)

[rsample](#)

[SuperLearner](#) - stratified CV

6. Hyperparameters tuning to find the best model configuration and improve the prediction on new data

- Optimize model settings to improve performance on new, unseen data
- Hyperparameters = settings that control how a model learns
(e.g., number of trees, tree depth, learning rate)
- Test a range of values rather than a single setting
- Use cross-validation to evaluate each configuration
- Select the configuration with the best validation performance
- Final model → train on all training data → evaluate on test data

7. Performance evaluation

- Most models output a **probability/score**
- If so, set a **cutoff** to transform **score to labels**
- Evaluate how well **predicted labels** match the **actual labels**

	c1	c2	c3	D				
Patient 4				no	Labelled Test set	Patient 4	0.1	cutoff=0.5
Patient 5				yes		Patient 5	0.45	

	P			True labels
Patient 4	0.1	10% confidence that the input belongs to class YES	no	no
Patient 5	0.45	45% confidence that the input belongs to class YES	no	yes

	YES	NO		True labels
Patient 4	0.1	0.9	no	no
Patient 5	0.45	0.55	no	yes

Count predicted and actual labels and evaluate

	D
Patient 6	yes
Patient 7	no
Patient 8	no
Patient 9	no
Patient 10	yes

	P	
Patient 6	0.8	yes
Patient 7	0.1	no
Patient 8	0.2	no
Patient 9	0.55	yes
Patient 10	0.45	no

cutoff=0.5

predicted	Positive	Negative
	TP	FN
actual	Positive	Negative
	FP	TN

predicted	Positive	Negative
	1	1
actual	Positive	Negative
	1	2

T: True; F: False
P: Positive; N: Negative

Confusion matrix and metrics

predicted	Positive	Negative
	1	1
actual	Positive	Negative
	1	2

		Actual class		
		Positive	Negative	
Predicted class	Positive	TP: True Positive	FP: False Positive (Type I Error)	Precision: $\frac{TP}{TP + FP}$
	Negative	FN: False Negative (Type II Error)	TN: True Negative	Negative Predictive Value: $\frac{TN}{TN + FN}$
		Recall or Sensitivity: $\frac{TP}{TP + FN}$	Specificity: $\frac{TN}{TN + FP}$	Accuracy: $\frac{TP + TN}{TP + TN + FP + FN}$

% of positive predictions that are actually correct

% of negative predictions that are actually correct

% of outcome correctly predicted

% of negative correctly predicted

% correctly predicted overall (e.g. yes and no)

TPR

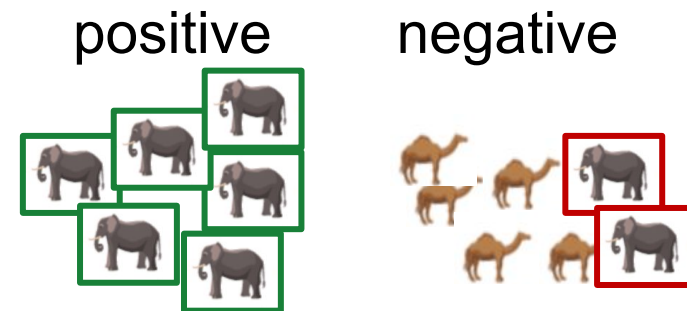
Metrics interpretation

		Actual class		
		Positive	Negative	
Predicted class	Positive	TP: True Positive	FP: False Positive (Type I Error)	Precision: $\frac{TP}{TP + FP}$
	Negative	FN: False Negative (Type II Error)	TN: True Negative	Negative Predictive Value: $\frac{TN}{TN + FN}$
Recall or Sensitivity:		$\frac{TP}{TP + FN}$	Specificity: $\frac{TN}{TN + FP}$	Accuracy: $\frac{TP + TN}{TP + TN + FP + FN}$

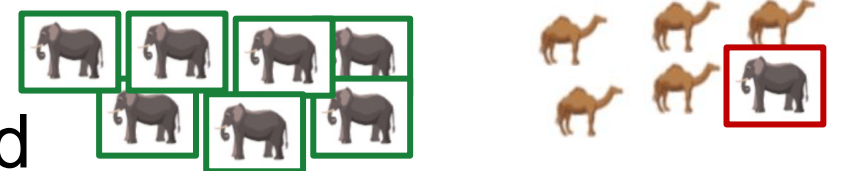


8 TP

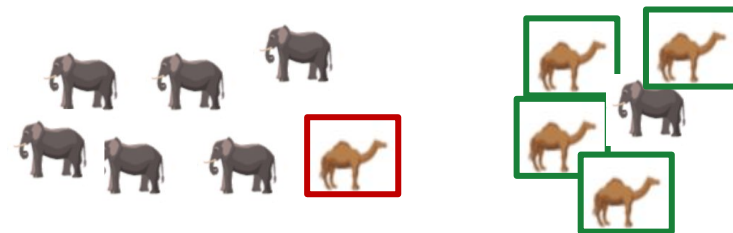
High precision - when the model predicts positive is usually right



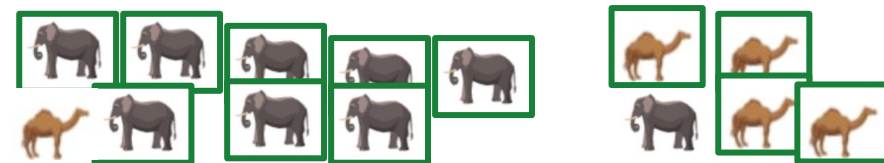
High recall - only few positive outcome are missed



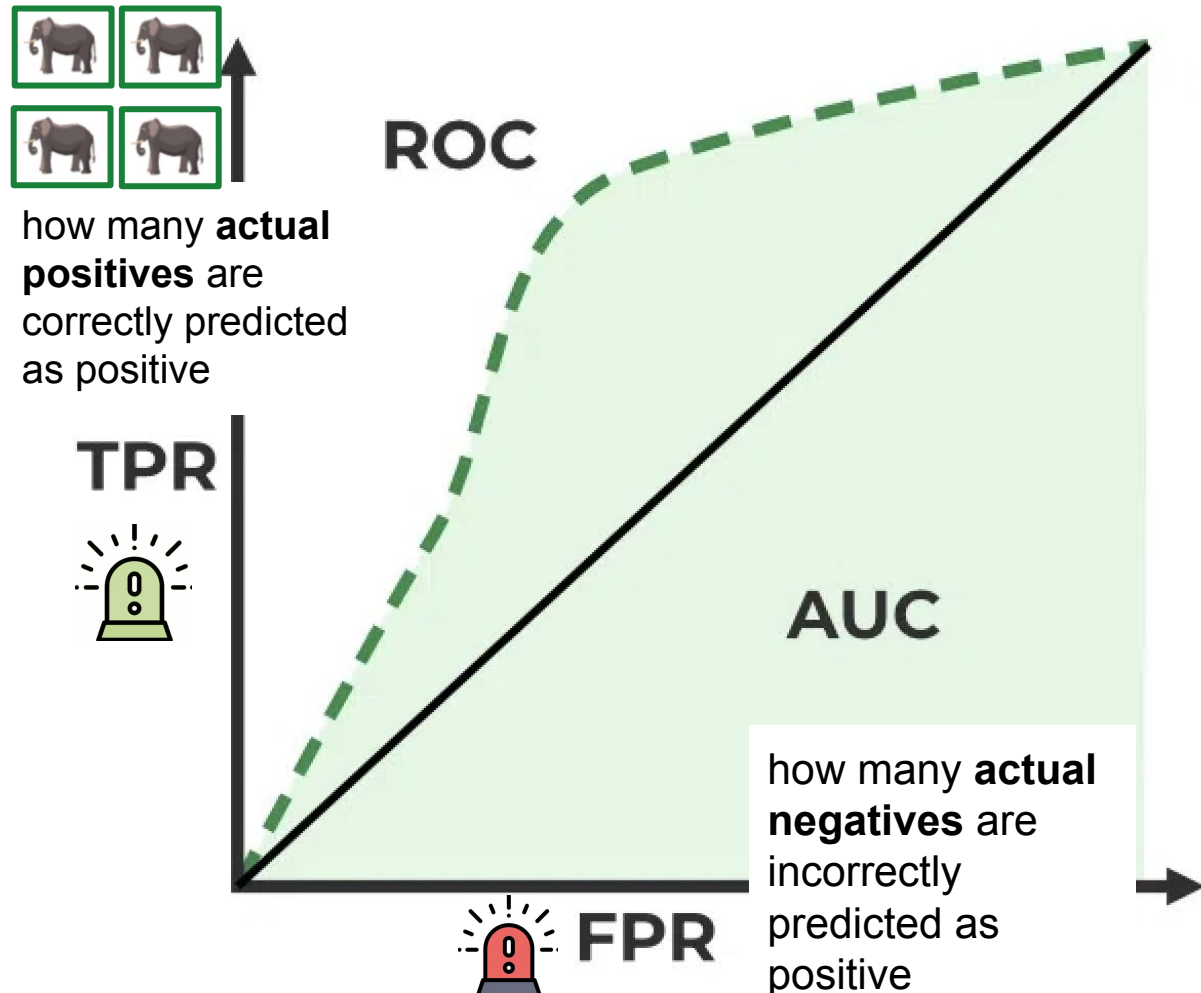
High specificity - very few false positive



High accuracy - most predictions are correct



ROC (Receiver Operating Characteristic) and AUC (Area Under the ROC Curve)



ROC - performance curve of TPR vs FPR across thresholds

AUC: good > 0.5 ; perfect = 1; poor < 0.5
How well the model classify / separate positive and negative classes

Break

See you in 5 minutes

Please fill the survey

<https://www.surveymonkey.com/r/bioinfo-training26>

ML algorithms

Poll 4. I have *behavioral measures* on a set of mice and I want to find subgroups of mice with similar behavior.

Which learning method will you apply?

1. Supervised learning method
2. Unsupervised learning method

Poll 5. I have a cohort of individuals with *variants information* and *disease status*.

I want to generate a machine learning model to predict disease status.

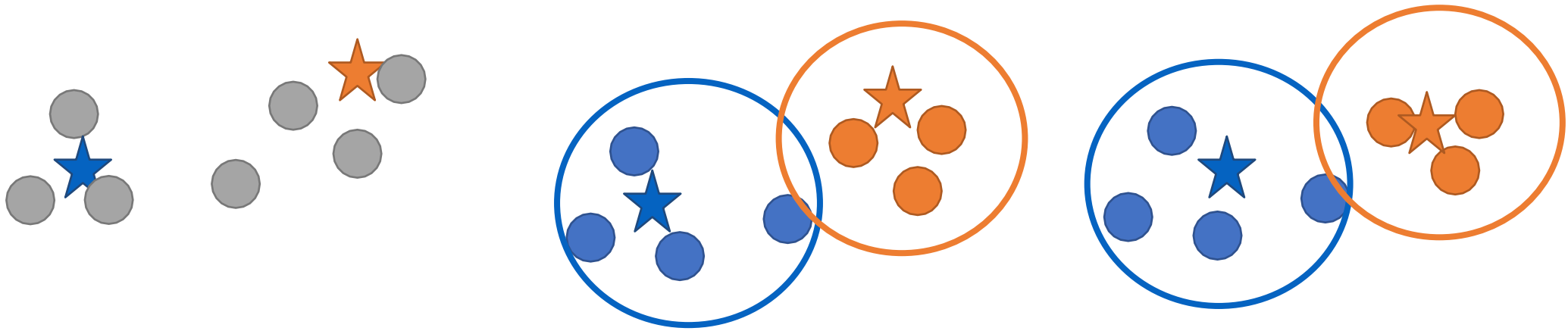
Which learning method will you apply?

1. Supervised learning method
2. Unsupervised learning method

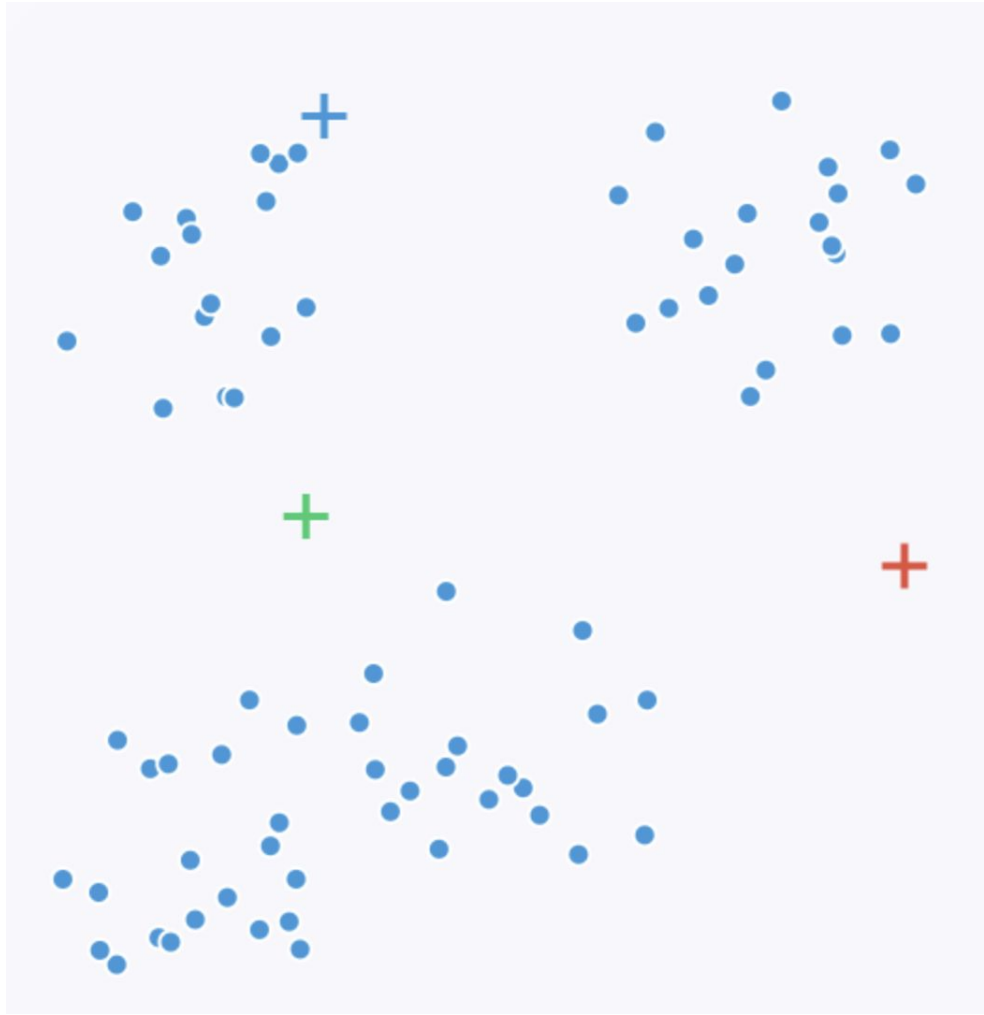
Unsupervised ML: K-means clustering steps

Partitioning data into **k clusters** by minimizing the **within-cluster sum of squared distances (Lloyd algorithm)**.

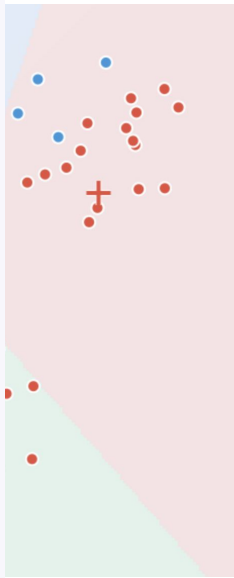
- Decide **k** number of clusters expected
- Start: Random centers (centroids)
- Take the sum of the squared distance between data points and all centers
- Assign a cluster membership to each data point
- Compute *new centers*
- End: if the centers don't change OR meet iteration threshold



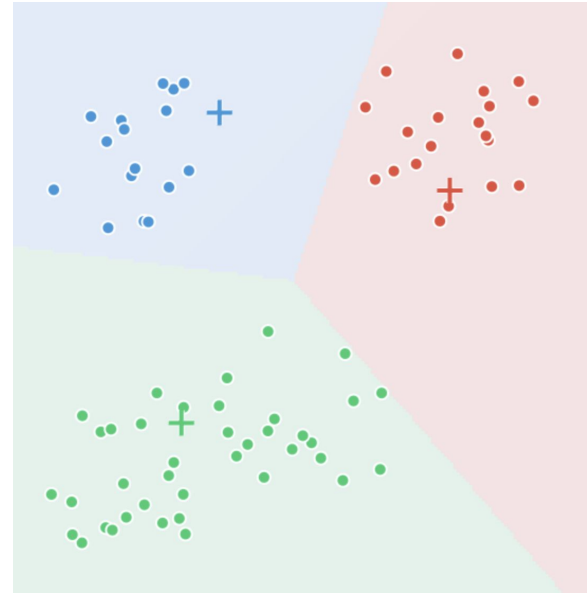
K-means clustering iterations



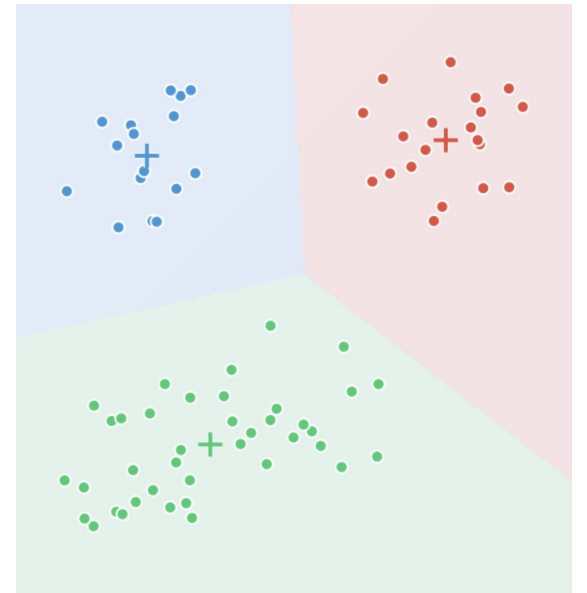
i=1



i=2



i=3



final setting

K-means clustering Pros and Cons

Pros:

- Simple and fast

Cons:

- Sensitive to the initial random selection of centers
- Computationally expensive
- Sensitive to the scale of data and outliers

Supervised ML: Decision Tree

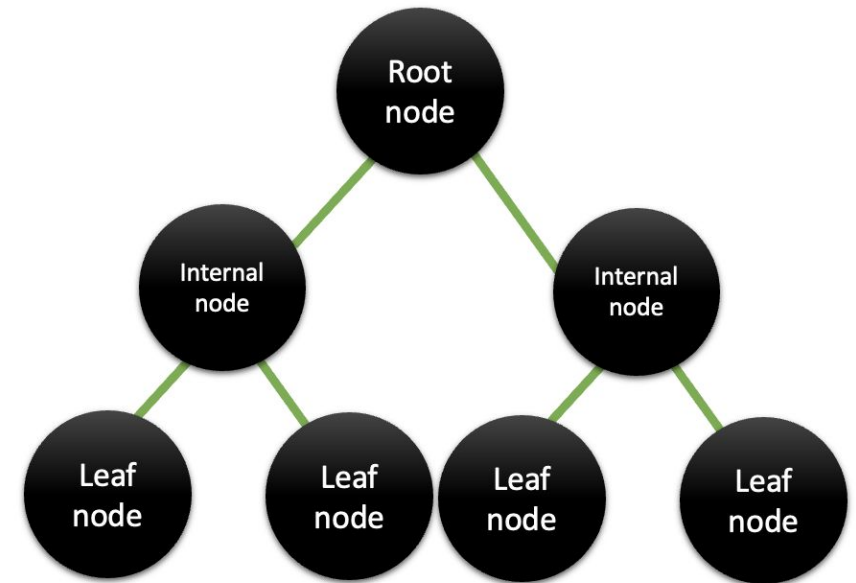
- Supervised approach
- Map of the possible outcomes of a series of related choices
- Splitting data based on features

Node: decision points based on features values

Leaf node: label - final predictions

Edge: connection between nodes - outcomes of the decision

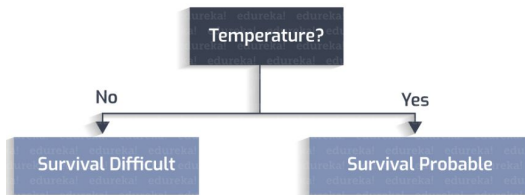
Decision tree



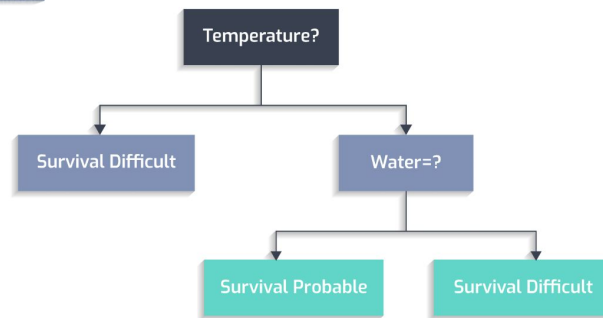
Decision tree answer a big question by a sequence of simple questions that lead to a clear decision

We have discovered a new planet - we ask the question whether would be possible to live there

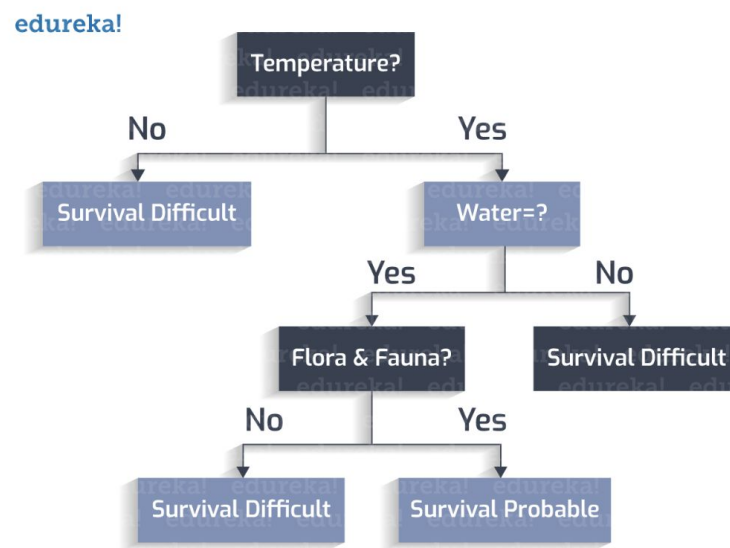
Range of survival temperature 0-100C



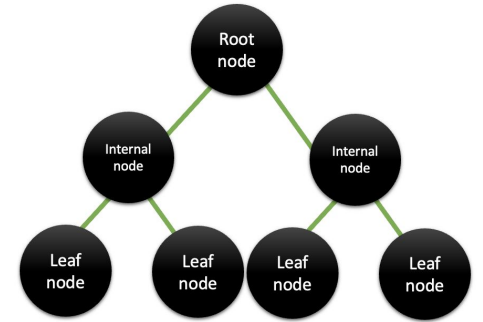
Is there water?



Are there animals/plants?



How does a decision tree work



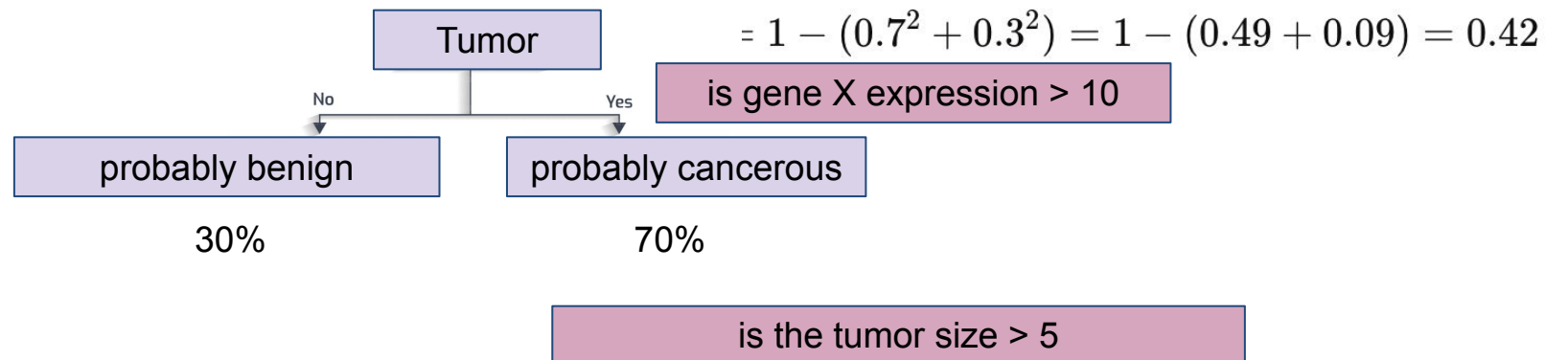
1. Start with **all data** at the root
2. Choose the **best feature** to split the data using metrics like:

- **Gini Impurity** - how impure a node is (0 pure)
$$\text{Gini} = 1 - \sum_{i=1}^C p_i^2$$
 proportion of class i in the node

1. Split the data into **subsets**.
2. Repeat recursively until:
 - All data in a node belongs to the same class, or
 - Maximum depth is reached.

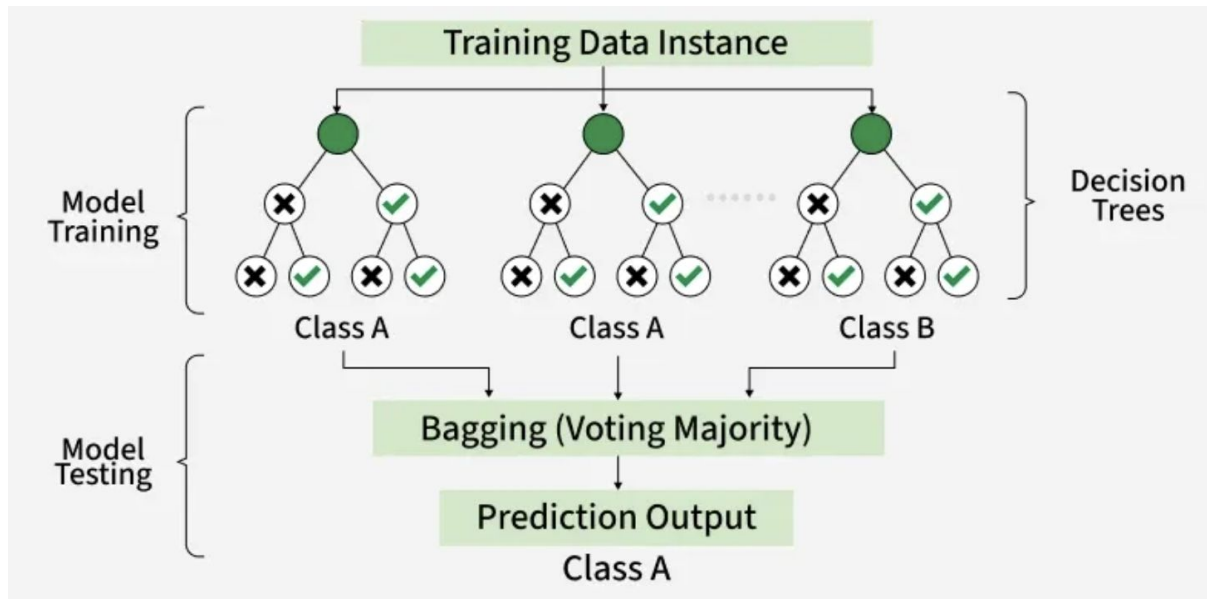
Decision tree example

Is a Tumor benign or cancerous



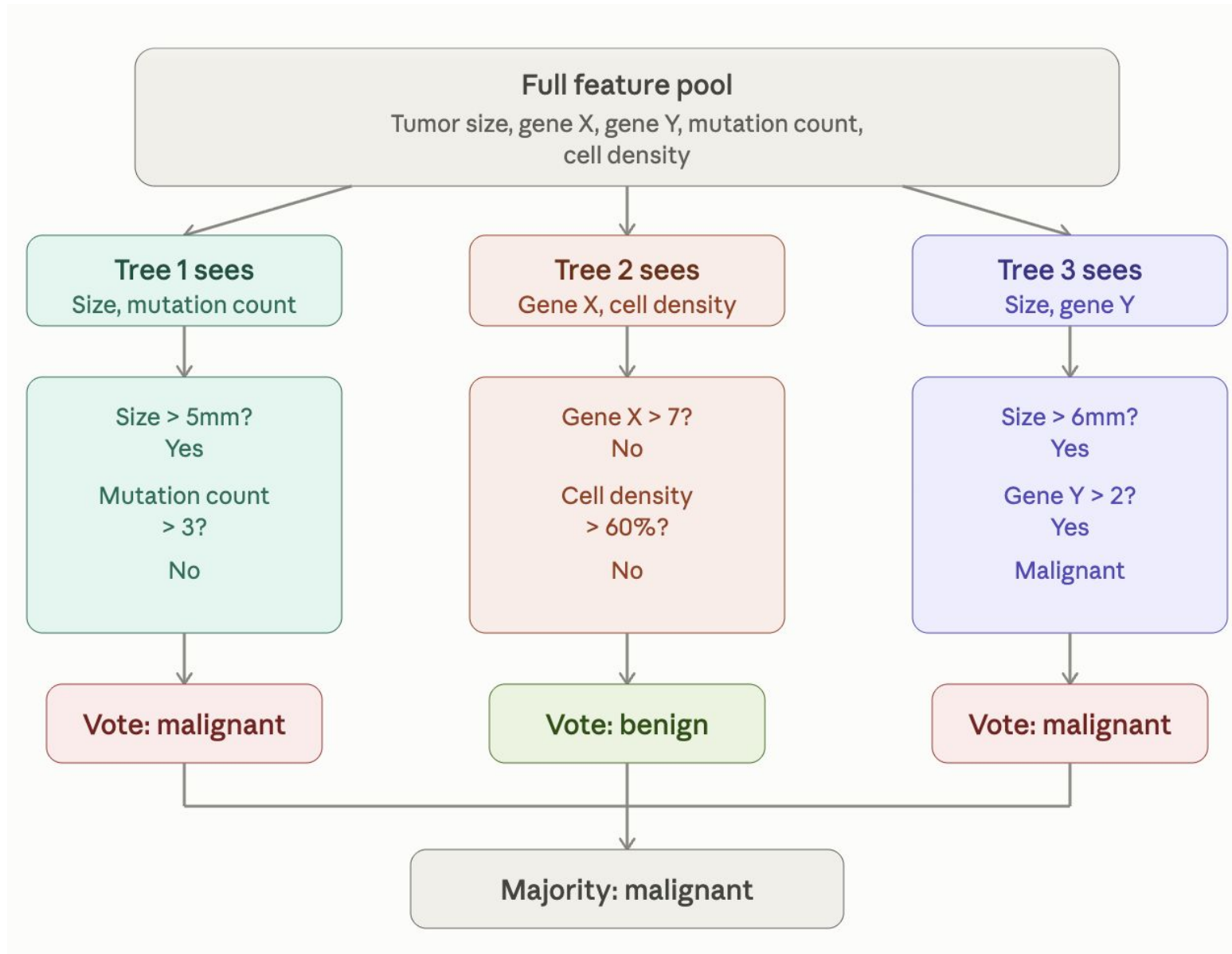
Random Forest

- **Random Forest** = Ensemble of many decision trees
- Uses **bagging (bootstrap aggregation)**: trains trees on **random samples** of data
- Randomly selects a subset of features at each split (**mtry** in R)
- Left out $\frac{1}{3}$ - out of bag samples



- Each tree votes; the majority vote is the **final prediction of oob samples**
- oob error rate - Reduces **overfitting** compared to a single tree
- More robust and accurate

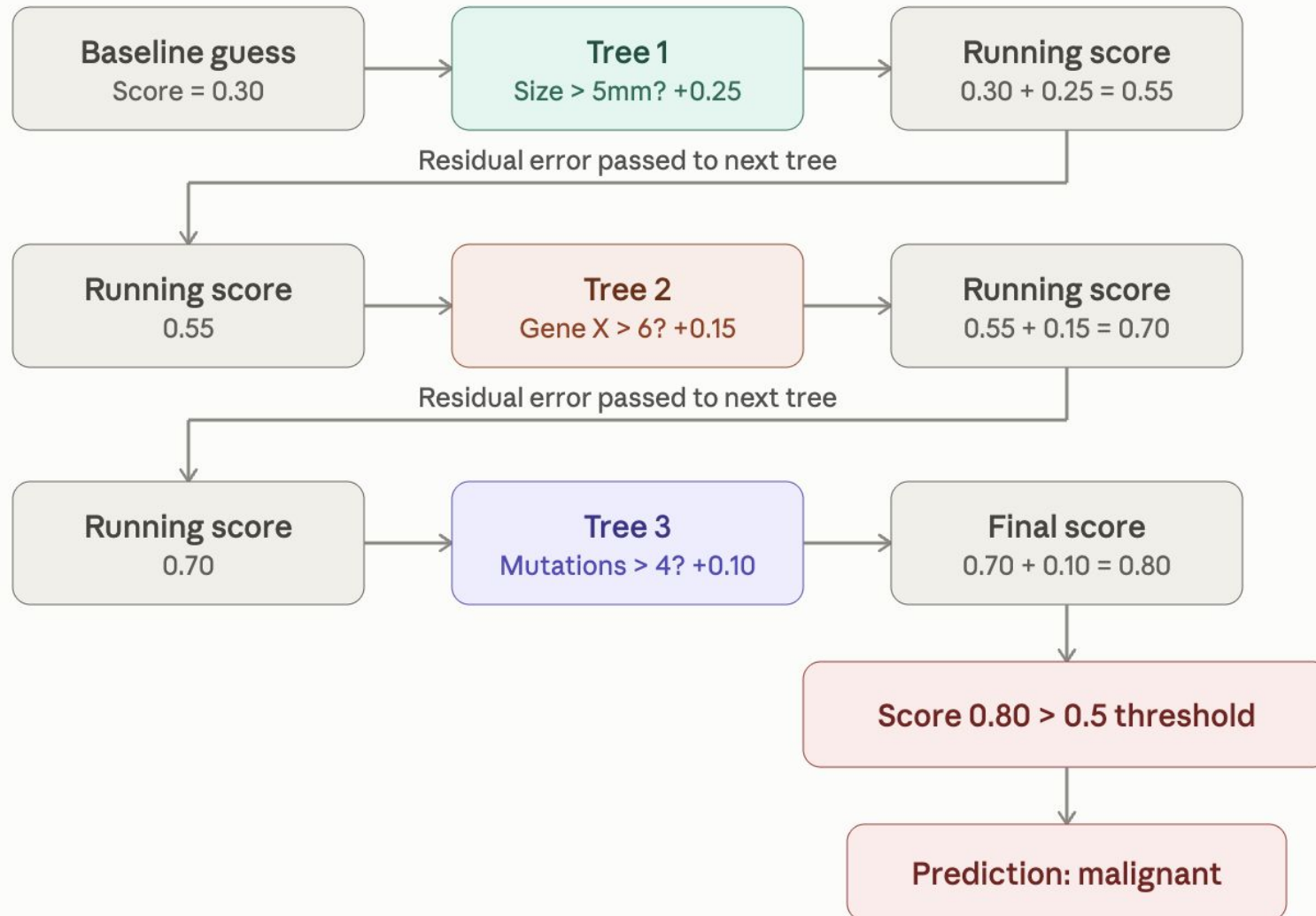
Random Forest



XGBoost (Extreme Gradient Boosting)

- **XGBoost** = trees built **sequentially**, each one correcting the previous tree's mistakes
- Uses **gradient boosting**: each new tree fits the **residual errors** of the trees before it
 1. Start with a naive baseline prediction (e.g., the overall malignancy rate across your training samples)
 2. Tree 1 is trained to fit the **residual errors** of that baseline (how far off is the current prediction, and in which direction?)
 3. Tree 2 is trained to fit whatever error is *still left* after Tree 1's correction
 4. This repeats iteratively
 5. Weighted sum each tree's contribution into a running score.
- Built-in **regularization** (L1/L2) and a **learning rate** help control overfitting
- Often more accurate than Random Forest, but has more parameters to tune

Xgboost



Random Forest vs XGBoost

Random Forest (Bagging)

- Trees trained in **parallel**, independently of each other
- Each tree trained on a **random bootstrap sample**
- Combined by **majority vote** / average
- Reduces **variance** by averaging over many trees
- Fast to train — trees can run in parallel
- Fewer hyperparameters to tune
- Great robust baseline, harder to squeeze out extra accuracy

XGBoost (Boosting)

- Trees trained **sequentially**, each depends on the last
- Each tree trained to fit the **residual errors (the loss gradient — exact for regression, approx for classification)**
- Combined by **weighted sum** (additive model)
- Reduces **bias** by targeting mistakes directly; controls variance via regularization
- Slower to train — trees depend on each other
- More hyperparameters (learning rate, depth, regularization)
- Often wins on accuracy, needs careful tuning to avoid overfitting

Break

See you in 5 minutes

Please fill the survey

<https://www.surveymonkey.com/r/bioinfo-training>

Where to find the algorithms in R and Python

Unsupervised methods:

- PCA (`stats::prcomp()` or `princomp()`, FactoMineR)
- **k-means** ([`stats::kmeans\(\)`](#))
- UMAP/ tSNE (`umap`, `Rtsne`, `Seurat`, etc)

Supervised methods:

- LDA (`MASS`)
- kNN (`caret`, `class`, ...)
- Naive Bayes (`e1071`, ..)
- **Random forest** ([`randomForest`](#), `ranger`, `SuperLearner`, ..)
- **Xgboost** ([`xgboost`](#))
- Support Vector Machine (`e1071`, ..)
- Neural Network (`nnet`, `keras`, ...)

Python packages and tutorials available in [scikit-learn](#)

Take-home messages

- Building a good supervised model is an iterative process: select features, manage bias/variance, tune hyperparameters, evaluate, and repeat
- The right features matter more than the number of features
- Balance bias and variance so that the model generalizes
- Tune parameters on validation data, never on the test set
- A model is only as good as its performance on data it has never seen

Your feedback is important to us!

Please take the survey ~3 min:

<https://www.surveymonkey.com/r/bioinfo-training26>

Upcoming workshops at Gladstone:

Introduction to RNaseq analysis

September 28-29

AI fluency for Biologists

October 5-6

Single Cell RNA-Seq Analysis using R

October 12-13

Introduction to Pathway Analysis

October 15

Statistics of Enrichment Analysis Methods

October 22-23

Linear Mixed Effects Models Using R

October 26-27

Hands-on

Dataset

- Coronavirus Disease 2019 (COVID-19) Clinical Data Repository: <https://covidclinicaldata.org/>
- A COVID-19 clinical dataset to introduce machine learning concepts. The dataset contains patient symptoms, vitals, and COVID-19 test outcomes.
- We use predictor variables (symptoms, comorbidities, vitals) to estimate the probability of an outcome variable (positive or negative COVID-19 test).
- Important note: Our focus is on building technical skills in machine learning. Although some symptoms or combinations of symptoms may correlate with test results, our goal is not to build a highly predictive model for COVID-19. Instead, we use this dataset to motivate exploration of ML techniques and concepts.

